🚀 Guia de Configuração - LIAS Dashboard Streamlit

📋 Índice

1. Instalação Rápida
2. Instalação Detalhada
3. Executar a Aplicação
4. Integração com Backend
5. Deploy
6. Troubleshooting

⚡ Instalação Rápida

```bash
# 1. Navegue até o diretório
cd lias-dashboard-streamlit

# 2. Instale as dependências
pip install -r requirements.txt

# 3. Execute a aplicação
streamlit run app.py
```

A aplicação abrirá automaticamente em `http://localhost:8501`

🔧 Instalação Detalhada

Passo 1: Verificar Python

```bash
python --version  # Deve ser 3.8 ou superior
```

Passo 2: Criar Ambiente Virtual (Recomendado)

No macOS/Linux:

```bash
python -m venv venv
source venv/bin/activate
```

No Windows:

```bash
python -m venv venv
venv\Scripts\activate
```

Passo 3: Instalar Dependências

```bash
pip install --upgrade pip
pip install -r requirements.txt
```

Passo 4: Verificar Instalação

```bash
streamlit --version
python -c "import pandas; import plotly; print('✅ Dependências instaladas!')"
```

🎯 Executar a Aplicação

Modo Desenvolvimento

```bash
streamlit run app.py
```

A aplicação estará disponível em: `http://localhost:8501`

Modo com Configurações Customizadas

```bash
```

```
streamlit run app.py --logger.level=debug
```

Modo com Porta Customizada

Bash

```bash
streamlit run app.py --server.port 8502
```

🔌 Integração com Backend

Opção 1: API REST

Edite `data.py` e adicione:

Python

```python
import requests
import os

API_URL = os.getenv("API_URL", "http://localhost:3000/api" )
API_KEY = os.getenv("API_KEY", "seu-token-aqui")

def get_logs_from_api(limit=50, offset=0):
    """Buscar logs de uma API REST"""
    try:
        response = requests.get(
            f"{API_URL}/logs",
            params={"limit": limit, "offset": offset},
            headers={"Authorization": f"Bearer {API_KEY}"},
            timeout=10
        )
        response.raise_for_status()
        return response.json()
    except requests.exceptions.RequestException as e:
        print(f"Erro ao buscar logs: {e}")
        return []

def create_log_entry(log_data):
    """Criar novo log via API"""
    try:
        response = requests.post(
            f"{API_URL}/logs",
            json=log_data,
```

```
            headers={"Authorization": f"Bearer {API_KEY}"},
            timeout=10
        )
        response.raise_for_status()
        return response.json()
    except requests.exceptions.RequestException as e:
        print(f"Erro ao criar log: {e}")
        return None
```

Depois, em `pages/logs.py`, substitua:

Python

```
# Antes
logs_df = get_logs_df(st.session_state.logs)

# Depois
from data import get_logs_from_api
logs_data = get_logs_from_api(limit=50)
logs_df = pd.DataFrame(logs_data)
```

Opção 2: MySQL/MariaDB

Instale o driver:

Bash

```
pip install mysql-connector-python
```

Edite `data.py`:

Python

```
import mysql.connector
import os

DB_CONFIG = {
    "host": os.getenv("DB_HOST", "localhost"),
    "user": os.getenv("DB_USER", "root"),
    "password": os.getenv("DB_PASSWORD", ""),
    "database": os.getenv("DB_NAME", "lias"),
}

def get_logs_from_db(limit=50, offset=0):
    """Buscar logs do banco de dados"""
    try:
```

```
        conn = mysql.connector.connect(**DB_CONFIG)
        cursor = conn.cursor(dictionary=True)
        cursor.execute(
            "SELECT * FROM logs ORDER BY data DESC LIMIT %s OFFSET %s",
            (limit, offset)
        )
        logs = cursor.fetchall()
        cursor.close()
        conn.close()
        return logs
    except mysql.connector.Error as err:
        print(f"Erro de banco de dados: {err}")
        return []

def create_log_entry_db(log_data):
    """Criar novo log no banco de dados"""
    try:
        conn = mysql.connector.connect(**DB_CONFIG)
        cursor = conn.cursor()
        query = """
        INSERT INTO logs
        (modelo, categoria, prompt, resposta, latencia, tokens, custo, status, 
        VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
        """
        cursor.execute(query, (
            log_data["modelo"],
            log_data["categoria"],
            log_data["prompt"],
            log_data["resposta"],
            log_data["latencia"],
            log_data["tokens"],
            log_data["custo"],
            log_data["status"],
            log_data["confianca"],
        ))
        conn.commit()
        cursor.close()
        conn.close()
        return True
    except mysql.connector.Error as err:
        print(f"Erro ao inserir log: {err}")
        return False
```

Opção 3: PostgreSQL

Instale o driver:

```
Bash
```

```
pip install psycopg2-binary
```

Edite `data.py`:

Python

```python
import psycopg2
import os

DB_CONFIG = {
    "host": os.getenv("DB_HOST", "localhost"),
    "user": os.getenv("DB_USER", "postgres"),
    "password": os.getenv("DB_PASSWORD", ""),
    "database": os.getenv("DB_NAME", "lias"),
}

def get_logs_from_db(limit=50, offset=0):
    """Buscar logs do PostgreSQL"""
    try:
        conn = psycopg2.connect(**DB_CONFIG)
        cursor = conn.cursor()
        cursor.execute(
            "SELECT * FROM logs ORDER BY data DESC LIMIT %s OFFSET %s",
            (limit, offset)
        )
        logs = cursor.fetchall()
        cursor.close()
        conn.close()
        return logs
    except psycopg2.Error as err:
        print(f"Erro de banco de dados: {err}")
        return []
```

🌐 Deploy

Deploy no Streamlit Cloud

1. **Crie uma conta em streamlit.io**
2. **Faça push do código para GitHub:**

Bash

```bash
git init
git add .
git commit -m "Initial commit"
git push origin main
```

1. **No Streamlit Cloud:**
   - Clique em "New app"
   - Selecione seu repositório
   - Escolha `app.py` como arquivo principal
   - Clique em "Deploy"

Deploy no Heroku

1. **Instale o Heroku CLI**
2. **Crie um arquivo `Procfile`:**

```
web: streamlit run --server.port=$PORT --server.address=0.0.0.0 app.py
```

1. **Crie um arquivo `.gitignore`:**

```
venv/
__pycache__/
*.pyc
.env
```

1. **Deploy:**

```bash
heroku create seu-app-name
git push heroku main
```

Deploy no Docker

1. **Crie um `Dockerfile`:**

```
```

```
FROM python:3.9-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

EXPOSE 8501

CMD ["streamlit", "run", "app.py", "--server.port=8501", "--server.address=0.0.
```

1. **Build e execute:**

Bash

```bash
docker build -t lias-dashboard .
docker run -p 8501:8501 lias-dashboard
```

🔐 Variáveis de Ambiente

Crie um arquivo `.env` na raiz do projeto:

Plain Text

```
# API Configuration
API_URL=http://localhost:3000/api
API_KEY=seu-token-aqui

# Database Configuration
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=sua-senha
DB_NAME=lias

# Streamlit Configuration
STREAMLIT_SERVER_HEADLESS=true
STREAMLIT_SERVER_PORT=8501
```

Depois, carregue no seu script:

Python

```python
from dotenv import load_dotenv
import os

load_dotenv( )

API_URL = os.getenv("API_URL")
API_KEY = os.getenv("API_KEY")
```

🐛 Troubleshooting

Erro: "No module named 'streamlit'"

```bash
pip install streamlit
```

Erro: "ModuleNotFoundError: No module named 'pages'"

Certifique-se de que existe um arquivo `pages/__init__.py`

Gráficos não aparecem

```bash
pip install --upgrade plotly streamlit
```

Porta 8501 já está em uso

```bash
streamlit run app.py --server.port 8502
```

Erro de conexão com banco de dados

- Verifique se o banco de dados está rodando
- Verifique as credenciais em `.env`
- Verifique a conectividade de rede

Aplicação lenta

- Reduza o número de logs exibidos
- Implemente cache com `@st.cache_data`
- Otimize as queries do banco de dados

📊 Estrutura de Banco de Dados

Se estiver usando um banco de dados, crie a seguinte tabela:

SQL

```sql
CREATE TABLE logs (
    id INT PRIMARY KEY AUTO_INCREMENT,
    data TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    modelo VARCHAR(50),
    categoria VARCHAR(100),
    prompt LONGTEXT,
    resposta LONGTEXT,
    latencia DECIMAL(5, 2),
    tokens INT,
    custo DECIMAL(8, 4),
    status ENUM('sucesso', 'erro', 'timeout'),
    confianca DECIMAL(3, 2),
    INDEX idx_modelo (modelo),
    INDEX idx_data (data),
    INDEX idx_status (status)
);
```

🎨 Personalização

Mudar Cores do Tema

Edite `.streamlit/config.toml`:

Plain Text

```
[theme]
primaryColor = "#FF6B6B"
backgroundColor = "#1E1E1E"
secondaryBackgroundColor = "#262730"
textColor = "#FFFFFF"
font = "sans serif"
```

Adicionar Logo

Edite `app.py`:

```python
st.set_page_config(
    page_title="LIAS Dashboard",
    page_icon="📊",
    layout="wide",
    initial_sidebar_state="expanded",
    menu_items={
        "Get Help": "https://seu-site.com/help",
        "Report a bug": "https://seu-site.com/bugs",
        "About": "# LIAS Dashboard v1.0.0"
    }
 )

st.sidebar.image("logo.png", width=200)
```

📚 Recursos Úteis

- Documentação Streamlit
- Plotly Python
- Pandas Documentation
- Python Requests

Última atualização: 2026-03-20

Versão: 1.0.0